Ders 5 Çalışma Özeti

Ders 5 Çalışma Özeti

Genel Konular

- İstatistiksel yöntemlere giriş (ekrandan, 00:10:00)
  - Slayttaki başlıklar: Prefix Kodlar, Shannon-Fano Coding, Huffman Coding, Arithmetic Coding, Diğer Yöntemler. Hoca diğer yöntemler altında faks sıkıştırmada kullanılan yöntemleri ve karakterleri tekli ya da ikili değil kombinasyonlar hâlinde ele alan PPM'i saydı.
  - Sıkıştırılan dosya geri açılabilmelidir; istatistiksel yöntemlerle çalışılıyorsa bunun için sembollere verilen kodların prefix özelliğine sahip olması gerekir.
- Sabit ve değişken uzunluklu kodlar (ekrandan, 00:20:00)
  - ASCII gibi sabit uzunluklu kodlar (her sembol 8 bit) sıkıştırma sağlamaz, çünkü veride artıklık (redundancy) kalır. Bu yüzden değişken uzunluklu kodlar seçilir; prefix kodlar da değişken uzunluklu kodlardandır.
  - Eşit olasılıklı dört sembol ($P_i = 0{,}25$): $H = -(4 \cdot 0{,}25 \log_2 0{,}25) = 2$ bit. `00 01 10 11` kodlarıyla $S = 2$, $R = |S - H| = 0$.
  - Farklı olasılıklar ($0{,}49$; $0{,}25$; $0{,}25$; $0{,}01$): $H = 1{,}57$ bit. Yine 2 bitlik sabit kod kullanılırsa $R = |2 - 1{,}57| = 0{,}43$. Olasılıklar farklıyken sabit uzunluklu kod artıklığı büyütür.
- Prefix olan ve olmayan kod örneği (ekrandan, 00:30:00)
  - Input: `a1 a3 a2 a1 a3 a3 a4 a2 a1 a1 a2 a2 a1 a1 a3 a1 a1 a2 a3 a1` (9 tane `a1`, 5 `a2`, 5 `a3`, 1 `a4`).
  - Code1: `a1=1`, `a2=01`, `a3=010`, `a4=001`. Code2: `a1=1`, `a2=01`, `a3=000`, `a4=001`.
  - `a1 a3 a1` Code1 ile `10101`, Code2 ile `10001` olur. Code1 çözülürken `01` okununca `a2` mi, bir bit daha okunup `a3` mü (`010`) olduğu bilinemez. Code2'de bu belirsizlik yoktur. Code1 prefix değildir, Code2 prefix koddur.
  - İki kodun bit uzunlukları aynı olduğu için ikisiyle de 20 sembol $9 \cdot 1 + 5 \cdot 2 + 5 \cdot 3 + 1 \cdot 3 = 37$ bitte kodlanır (ASCII ile $20 \cdot 8 = 160$ bit). $S = 37/20 = 1{,}85$ bit; slaytta $H = 1{,}57$ ve $R = 0{,}28$ yazılır.
- Prefix kodun tanımı
  - Bir sembolün kodunun bit deseni başka bir sembolün kodunun ön eki (prefix) olamaz. Bütün istatistiksel yöntemlerin ürettiği kodlar prefix olmalıdır.
  - Değişken uzunluklu kod oluşturulurken iki kural: en sık kullanılan sembole en kısa kod atanır ve kod prefix özelliğine uymalıdır.
  - Tam sayıların binary gösterilimi (`0`, `1`, `10`, `11`, `100`, ...) prefix değildir; örneğin `1`, `10` ve `11`'in ön ekidir. Hepsi aynı bit sayısıyla yazılırsa prefix olur ama bu kez kod sabit uzunluklu olur.
- Hazır prefix kod örneği: başa $\lfloor \log_2 i \rfloor$ sıfır eklemek (ekrandan, 00:44:40 ve 00:47:20)
  - $i \geq 1$ için i'nin binary değerinin başına $\lfloor \log_2 i \rfloor$ adet sıfır eklenir: 1 → `1`, 2 → `010`, 3 → `011`, 4 → `00100`, 5 → `00101`, 8 → `0001000`, 15 → `0001111`, 16 → `000010000`.
  - Kod uzunluğu $1 + 2\lfloor \log_2 i \rfloor$ bittir: $i = 16$ için 9 bit, $i = 6$ için 5 bit.
  - $\lfloor x \rfloor$ kendisinden küçük ya da eşit en büyük tam sayı, $\lceil x \rceil$ kendisinden büyük ya da eşit en küçük tam sayıdır ($\lceil 7{,}2 \rceil = 8$, $\lfloor 7{,}2 \rfloor = 7$).
  - Olasılıklar büyükten küçüğe sıralanıp kodlar sırayla verilir. Ancak hazır kodlar verinin içeriğinden bağımsız olduğu için artıklık genelde yüksek kalır.
- Unary kod ve start-step-stop (ekrandan, 01:10:00 ve 01:14:40)
  - n 1'den başlıyorsa unary kod $(n-1)$ adet 1 ve onu takip eden bir 0'dır (ya da tersi: $(n-1)$ adet 0 ve bir 1). n 0'dan başlıyorsa n adet 1 ve bir 0 yazılır. Örnek: 1 → `0`, 2

→ `10`, 3 → `110`, 4 → `1110`, 5 → `11110`; alternatif kod `1`, `01`, `001`, `0001`, `00001`.
  - Unary kodlar "start-step-stop" kodu olarak da bilinir: $a = \text{start} + n \cdot \text{step}$. $a = \text{stop}$ olduğunda unary kısmın sonundaki 0 düşürülür.
  - 680 kod için triplet $(3, 2, 9)$: $n = 0, 1, 2, 3$ için $a = 3, 5, 7, 9$; kod sözcükleri `0xxx`, `10xxxxx`, `110xxxxxxx`, `111xxxxxxxxx`; sayıları $2^3 = 8$, $2^5 = 32$, $2^7 = 128$, $2^9 = 512$; aralıklar 0-7, 8-39, 40-167, 168-679.
  - Kod sayısı: $\frac{2^{\text{stop}+\text{step}} - 2^{\text{start}}}{2^{\text{step}} - 1} = \frac{2^{11} - 2^3}{2^2 - 1} = 680$. Sayı stop parametresine bağlı olarak üstel artar.
- C1, C2, C3, C4 prefix kodları (ekrandan, 01:20:00)
  - $B(n)$: n'nin binary karşılığı. $|B(n)|$: binary sayının dijit uzunluğu. $\bar{B}(n)$: en soldaki bit (her zaman 1) atıldığında kalan bitler.
  - **C1** (ekrandan, 01:22:50 ve 01:28:40): $|B(n)|$'nin unary kodu ile $\bar{B}(n)$'nin birleşimi. $C_1(16) = 11110|0000$, $C_1(12) = 1110|100$, $C_1(6) = 110|10$. Uzunluk $2\lfloor \log_2 n \rfloor + 1$ bit; $n = 32$ için 11 bit.
  - **C2** (ekrandan, 01:32:45): C1'in birinci bölümünden bir bit, ikinci bölümünden bir bit sırayla alınarak yazılır. $C_2(16) = 101010100$. Uzunluğu C1 ile aynıdır.
  - **C3** (ekrandan, 01:35:50): $|B(n)|$'nin C2 kodu, arkasına $\bar{B}(n)$. $C_3(15) = C_2(4)|\bar{B}(15) = 10100|111$, $C_3(10) = 10100|010$. Uzunluk $1 + \lfloor \log_2 n \rfloor + 2\lfloor \log_2(1 + \lfloor \log_2 n \rfloor) \rfloor$ bit; $n = 15$ için 8 bit.
  - **C4** (ekrandan, 01:40:00 ve 01:42:45): önce $B(n)$ yazılır; soluna $|B(n)| - 1$'in binary gösterilimi eklenir; işlem eklenen değer iki dijit kalıncaya kadar sürer; en sonda sağ tarafa 0 yazılır. $C_4(16) = 10|100|10000|0$, $C_4(64) = 10|110|1000000|0$.
- Golomb kodu (ekrandan, 01:50:00 ve 01:52:30)
  - Unary kodun özelliklerini kullanan basit bir prefix koddur. Karakterler frekans ya da olasılıklarına göre büyükten küçüğe sıralanır, 0'dan başlayan ardışık x değerleri atanır, ideali 2'nin kuvveti olan bir M seçilir.
  - $x_M = \lfloor x/M \rfloor$, $x_L = x \bmod M$, $x = x_M \cdot M + x_L$. Golomb kodu x_M'nin unary kodu ile x_L'nin binary değerinin birleşimidir.
  - $M = 4$, dokuz karakter (`a e k l m n i y b`): `000`, `001`, `010`, `011`, `1000`, `1001`, `1010`, `1011`, `11000`.
  - İlk dört karakter aynı uzunlukta (3 bit) kodlanır; olasılıklar çok farklıysa (örneğin 0,7 ve 0,3) bu seçim artıklığı yükseltir.
- Hazır kodlar ile yöntem ürünü kodlar
  - Buraya kadarki kodlar önceden hazırlanmış tablolardır ve verinin içeriğinden bağımsızdır. Bundan sonraki istatistiksel yöntemler kodları verinin içeriğine göre oluşturur ve hepsi prefix özelliğine sahiptir.
- Shannon-Fano kodlama (ekrandan, 02:10:00)
  - Değişken uzunluklu bir yöntemdir. Olasılık ya da kullanım sıklıkları bilinen n elemanlı bir kümeyle başlanır; semboller azalan sırada sıralanır.
  - Küme aynı ya da yakın olasılık toplamlarına göre iki kümeye ayrılır; bir kümedeki sembollere 1, diğerine 0 verilir. Her alt küme kendi içinde bölünmeye devam eder. Kodlar oluşan ağaç üzerinde okunur.
  - Olasılıklar 2'nin negatif kuvvetleri ($1/2$, $1/4$, $1/8$, ...) olduğunda en iyi sonucu verir; gerçek veride bu pek olmaz.
- Shannon-Fano örneği (ekrandan, 02:20:00 ve 02:22:40)
  - Olasılıklar $0{,}25$; $0{,}20$; $0{,}15$; $0{,}15$; $0{,}10$; $0{,}10$; $0{,}05$ (semboller `a e k m l n z`). İlk bölme $0{,}45/0{,}55$.
  - Kodlar: `a=11`, `e=10`, `k=011`, `m=010`, `l=001`, `n=0001`, `z=0000`.
  - $H = 2{,}67$ bit, $S = 0{,}25 \cdot 2 + 0{,}20 \cdot 2 + 3 \cdot 0{,}15 \cdot 2 + 3 \cdot 0{,}10 + 4 \cdot 0{,}10 + 4 \cdot 0{,}05 = 2{,}7$ bit, $R = 0{,}03$ bit.
  - `kale` → `011 11 001 10` = `0111100110` (10 bit). Çözmede bit bit okunur; prefix özelliği sayesinde eşleşen kod bulunduğu anda sembol kesinleşir.
- İkinci Shannon-Fano örneği (ekrandan, 02:35:30)
  - Olasılıklar $0{,}4$; $0{,}4$; $0{,}1$; $0{,}05$; $0{,}05$. Bölmeler $0{,}4/0{,}6$, sonra $0{,}4/0{,}2$, sonra $0{,}1/0{,}1$, sonra $0{,}05/0{,}05$. Kodlar `1`, `01`, `001`, `0001`, `0000`.

Hocanın Özellikle Vurguladığı Kısımlar

- Prefix özelliği olmazsa olmazdır
  - Kodlanan doküman geri açılamıyorsa kodun "hiçbir esprisi yoktur". Prefix olmayan kodla açarken ya bir yerde takılınır ya da sonuna kadar gidilip bambaşka bir içerik (örneğin `kale` yerine `kaze`) elde edilebilir.
- Bitleri bayt olarak yazmamak
  - Geçmişte ödevlerde yapılan bir hata: her 0 ve 1 dosyaya birer bayt olarak yazılıyor. `kale` 10 bitlik koduyla 10 bayt olur, orijinali 4 bayttır; bu sıkıştırma değil genişletmedir. Bitler kaydırılarak (shift) baytların içine yerleştirilmelidir; `kale` yaklaşık 2 bayt tutar.
- Hazır kodlarda artıklık
  - Hazır kodlar verinin dağılımına bakmadan kullanılırsa olasılıkları birbirine çok yakın iki sembolden biri 1, diğeri 3 bitle kodlanabilir ya da çok farklı olasılıklar aynı bitle kodlanabilir. Önce verinin dağılımına bakıp ona uygun kod seçmek gerekir.
- Alıcının kodları bilmesi
  - Sıkıştırılmış verinin açılabilmesi için başına bir header eklenir (olasılıklar, sembol kodları ya da başka bir bilgi). Bunun ayrıntıları Huffman'da anlatılacak.
- Kayıpsız sıkıştırmanın doğrulanması
  - Kayıpsız sıkıştırmada açılan dosya orijinalle bayt bayt karşılaştırıldığında birebir aynı çıkmalıdır. Kayıplı sıkıştırmada bayt bayt aynılık beklenmez; görüntüde başarı root mean square ya da sinyal-gürültü oranıyla ölçülür.

Kısa Tekrar Notları

- Sabit uzunluklu kod → artıklık; değişken uzunluklu ve prefix kod gerekir.
- $\{0{,}25\} \times 4$: $H = 2$, $S = 2$, $R = 0$. $\{0{,}49; 0{,}25; 0{,}25; 0{,}01\}$ sabit 2 bit: $R = 0{,}43$.
- Prefix: hiçbir kod başka bir kodun ön eki değildir. Code1 (`1, 01, 010, 001`) prefix değil; Code2 (`1, 01, 000, 001`) prefix.
- Tam sayıların değişken uzunluklu binary yazımı prefix değildir.
- Başa $\lfloor \log_2 i \rfloor$ sıfır: uzunluk $1 + 2\lfloor \log_2 i \rfloor$.
- Unary: $(n-1)$ adet 1 + 0; n 0'dan başlarsa n adet 1 + 0.
- Start-step-stop: $a = \text{start} + n \cdot \text{step}$; $a = \text{stop}$ ise sondaki 0 düşer; $(3, 2, 9)$ → 680 kod.
- Kod sayısı $\dfrac{2^{\text{stop}+\text{step}} - 2^{\text{start}}}{2^{\text{step}} - 1}$.
- C1 = unary($|B(n)|$) + $\bar{B}(n)$; C2 = C1'in iki bölümünün bitleri sırayla; C3 = C2($|B(n)|$) + $\bar{B}(n)$; C4 = $B(n)$'nin soluna $|B| - 1$ değerleri, sonda 0.
- Golomb: $x_M = \lfloor x/M \rfloor$ unary, $x_L = x \bmod M$ binary; $M = 4$ için `000 ... 011`, `1000 ... 1011`, `11000`.
- Shannon-Fano: azalan sırala, yakın toplamlı iki kümeye böl, 1/0 ver, tekrarla.
- Örnek: `11, 10, 011, 010, 001, 0001, 0000`; $H = 2{,}67$, $S = 2{,}7$, $R = 0{,}03$; `kale` = `0111100110`.

Detaylı Açıklamalar

- **Code1 neden açılamaz?** `10101` dizisinde ilk `1` kesin olarak `a1`'dir, çünkü 1 ile başlayan başka kod yoktur. Ardından `0` gelir; tek bitlik `0` kodu yoktur. `01` okununca `a2` bulunur, ama `010` (`a3`) de mümkündür. Hangi yolun seçileceği bilinemez; bir yol seçilip devam edilirse metnin sonunda yanlış bir içerik çıkabilir ya da bir noktada hiçbir kodla eşleşmeyip takılınır. Code2'de `10001` okunurken `1` → `a1`, ardından `0` ve `00` ile başlayan birden çok kod olduğu için üçüncü bit okunur; `000` yalnız `a3`'tür; son `1` → `a1`. Hiçbir an belirsizlik yoktur.
  - Ders sonrası not: Hoca 00:28:43'te `a4`'ü 1 bit sayıp toplamı tutturamadı, sonra `a4`'ün 3 bit olduğunu söyleyip 37 bite ulaştı (slaytta `1 bit` üstü çizilip 3 yazıldı).
  - Ders sonrası not: Slayttaki tabloda olasılıklar $0{,}49$; $0{,}25$; $0{,}25$; $0{,}02$ yazıyor (toplam 1,01) ve $H = 1{,}57$ önceki örnekten alınmış. Input'taki sayımlara göre olasılıklar $9/20 = 0{,}45$, $5/20 = 0{,}25$, $5/20 = 0{,}25$, $1/20 = 0{,}05$'tir; bu dağılımın entropisi python ile yaklaşık $1{,}73$ bit çıkar, artıklık da $|1{,}85 - 1{,}73| \approx 0{,}11$ bit olur. Slayttaki $0{,}28$, $H = 1{,}57$ alınarak hesaplanmıştır.

- **Başa sıfır ekleyerek prefix kod.** $i = 5$: binary `101`; $\log_2 5$ 2 ile 3 arasındadır, $\lfloor \log_2 5 \rfloor = 2$; başa iki sıfır → `00101`. $i = 15$: binary `1111`; $\log_2 15 \approx 3{,}9$, taban 3; → `0001111`. Uzunluk formülü $1 + 2\lfloor \log_2 i \rfloor$: $i = 16 \to 1 + 2 \cdot 4 = 9$; $i = 6 \to 1 + 2 \cdot 2 = 5$. Bu formülle tabloyu kurmadan, örneğin 64. kodun uzunluğu doğrudan bulunur.
  - Ders sonrası not: Hoca 00:46:37'de 16 için "eşit ve kendisinden büyük integer sayı 4" dedi; formüldeki işaret alt tamdır (kendisinden küçük ya da eşit en büyük tam sayı). $\log_2 16 = 4$ tam sayı olduğu için sonuç değişmez.
- **Start-step-stop'ta sıfırın düşürülmesi.** Unary kısımlar `0`, `10`, `110` ve son grup için `1110` olmalıydı. Ancak $a =$ stop olan son gruptan sonra başka grup gelmeyeceği için `111` ile başlayan ve arkasından başka bir şey gelen kod yoktur; sondaki 0 gereksizdir ve düşürülür. Önlerine unary kısım konan binary sayılar (3, 5, 7, 9 bitlik) bu sayede prefix olur: hiçbir kod `0`, `10`, `110` ya da `111` ile başlayan başka bir grubun koduyla karışmaz.
  - Ders sonrası not: Hoca 01:16:13'te üç grup için "160 tane" dedi, sonra "167 tane" dedi; $8 + 32 + 128 = 168$ koddur (0-167 aralığı). Dört grupla $168 + 512 = 680$ olur.
- **C1-C4 örneklerinin çözümü.** $C_1(12)$: $B(12) = 1100$, $|B(12)| = 4$, unary(4) = `1110`, $\bar{B}(12) = 100$ → `1110100`. $C_1(6)$: $B(6) = 110$, $|B| = 3$, unary(3) = `110`, $\bar{B} = 10$ → `11010`. $C_2(16)$: C1'in bölümleri `11110` ve `0000`; bitler sırayla `1,0,1,0,1,0,1,0,0` → `101010100`. $C_3(10)$: $|B(10)| = 4$, $C_2(4) = 10100$, $\bar{B}(10) = 010$ → `10100010`. $C_4(64)$: $B(64) = 1000000$ (7 dijit); $7-1 = 6$ → `110` (3 dijit); $3-1 = 2$ → `10` (2 dijit, dur); sonda 0 → `10|110|1000000|0`. Bütün kodlar slayttaki tabloyla python ile karşılaştırıldı ve tuttu.
  - Ders sonrası not: Hoca $C_4(64)$'te önce 6 yerine 12 yazdı ve sondaki 0'ı en sola koydu (01:41:35-01:42:31); kendisi düzeltti. Kural slayta göre: sıfır en son adımda sağ tarafa yazılır.
- **C kodlarının karşılaştırılması.** C1'de en olası sembol 1 bitle, sonraki ikisi 3 bitle, sonraki dördü 5 bitle kodlanır. C2'yi elde etmek için önce C1, C3'ü elde etmek için önce C2 bilinmelidir; C4 ise ötekilerden bağımsızdır. Hoca bu tablolar sayesinde, örneğin 120 sembol için en uzun kodun kaç bit olacağının tabloya bakılarak hemen söylenebileceğini belirtti.
- **Golomb kodunda artıklık.** $M = 4$ seçildiğinde her x_M grubunda 4 kod aynı uzunluktadır: ilk dört karakter 3 bit, sonraki dört karakter 4 bit, dokuzuncu karakter 5 bit. `a`'nın olasılığı 0,7, `e`'nin 0,3 gibi çok farklı olsaydı ikisini de 3 bitle kodlamak doğru bir seçim olmazdı. Olasılıklar birbirine yakınsa eşit uzunlukta kodlamak artıklığı fazla artırmaz.
- **Shannon-Fano'da bölme.** Örnekte toplam 1'dir; tam $0{,}5/0{,}5$ bölme yoktur. İlk iki sembolde bölünürse $0{,}45/0{,}55$, üç sembolde $0{,}60/0{,}40$ olur; $0{,}45/0{,}55$ en dengelisidir. Sonra üst küme $0{,}25/0{,}20$, alt küme $0{,}30/0{,}25$ ($0{,}15 + 0{,}15$ ile $0{,}10 + 0{,}10 + 0{,}05$), $0{,}15/0{,}15$, $0{,}10/0{,}15$ ve en son $0{,}10/0{,}05$ diye bölünür. Her bölmede üst kümeye 1, alt kümeye 0 verilir. İkinci örnekte $0{,}8/0{,}2$ yerine $0{,}4/0{,}6$ seçilir; amaç ulaşılabilen en yakın iki toplamdır.
  - Ders sonrası not: Hoca 02:11:35'te sıralamayı "en küçükten en büyüğe doğru" diye söyledi; slayta ve örneğe göre semboller azalan sırada (büyükten küçüğe) sıralanır.
  - Ders sonrası not: Hoca S'yi hesaplarken 02:27:58'de önce iki 0,15'i ve iki 0,10'u 3 bit saydı, sonra düzeltti: 0,10'lardan biri 3, diğeri 4 bittir. Slayttaki $S = 2{,}7$ doğrudur; H python ile $2{,}666 \approx 2{,}67$ bit çıktı.
  - Ders sonrası not: 02:30:50'de altyazıda "kayıplı sıkıştırma yapmıyorsam, kayıpsız sıkıştırma yapıyorsam bu ikisi birbirinden farklı olabilir" geçiyor; hocanın hemen sonraki cümlelerine göre kastedilen, kayıpsızda iki dosyanın birebir aynı, kayıplıda farklı olabileceğidir.
- **İkinci Shannon-Fano örneğinin değerlendirmesi.** Derste yalnız kodlar çıkarıldı. Python ile: $S = 0{,}4 \cdot 1 + 0{,}4 \cdot 2 + 0{,}1 \cdot 3 + 0{,}05 \cdot 4 + 0{,}05 \cdot 4 = 1{,}9$ bit, $H \approx 1{,}82$ bit, $R \approx 0{,}08$ bit (derste hesaplanmadı).
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.